

How Far Can Prompting Go?

A Multi-Agent Socratic Math Tutor with Deterministic Verification

September 20, 2026

Abstract

This report presents a prompt-driven multi-agent Socratic tutoring system for mathematics and examines the limits of relying primarily on large language models for dialogue control, state tracking, and correctness judgements. The system combines an orchestrator, a mathematical reasoning agent, a metacognitive support agent, and a lightweight learner-support signal, and has recently been extended with a deterministic SymPy-based verification component. This report describes the current architecture, the key design tensions, observed failure modes, design and results of an ablation test that compared single-agent, multi-agent, and multi-agent + verification configurations. The test results revealed that multi-agent systems surpass single-agent system in error detection, mathematical guidance, metacognitive timing and session close; whereas single-agent system was stronger at remaining Socratic and resilient at state regression. The multi-agent and multi-agent + verification conditions produced similar outcomes due to the limitations of the verifier parser.

1 Problem and Research Questions

Large Language Model (LLM)-based tutoring systems show potential in personalised dialogue-based tutoring (1). While the single-model-based systems can adapt and generate personalised response in real time, it faces limitations in handling mathematical reasoning, dialogue management, metacognitive scaffolding and affective adaption in parallel. Additionally, single-model math tutors often lack pedagogical focus, reveal solutions too early and accept student steps incorrectly(1; 2; 3). These systems also struggle with consistent dialogue state tracking, which is examined below.

Multi-agent system architecture can address the aforementioned issues by assigning distinct pedagogical functions to specialised agents. In this project, we implement and examine a prompt-driven multi-agent Socratic math tutor that comprises:

1. a mathematical reasoning agent
2. a metacognitive support agent
3. a lightweight learner-support (affective) signal
4. an orchestrating agent for coordination, and
5. a deterministic SymPy-based verification component

The system is designed to answer the following research questions, which will be investigated through controlled comparisons:

RQ1: Does a multi-agent architecture improve mathematical correctness and Socratic restraint compared to a carefully prompted single-agent tutor?

RQ2: Does the addition of deterministic symbolic verification further reduce incorrect acceptance of student steps compared with the multi-agent system without verification?

2 System Architecture

The backend of the system is organised as a multi-agent tutoring pipeline built with LangGraph, exposed by a lightweight FastAPI layer. A Chainlit frontend is designed to handle interaction and visualisation. At a high level:

Student message → State update & verification signal → Affective signal → Orchestrator decision → Specialist response → Final polished message

The design deliberately keeps most coordination inside prompts and a relatively simple graph structure, rather than a heavy deterministic controller. Four agents are designed for different tasks:

- **Orchestrating Agent** (Qwen3.5-122B-A10B-4bit): the central decision-making agent. It tracks the current problem and recent history, decides whether the next step should be tutoring or reflection, writes a focused instruction for the chosen agent, judges when the problem appears solved. In practice, the Orchestrator mainly routes between continued tutoring and post-solution reflection. Free mid-process calls to the Metacognitive agent were restricted for stability reasons (see below).
- **Mathematical Reasoning Agent** (Qwen3.5-122B-A10B-4bit): The main Socratic tutoring agent. It receives a narrow instruction from the Orchestrator, produces the next micro-step or guiding question, avoids giving full solutions.
- **Metacognitive Support Agent** (Qwen3.5-122B-A10B-4bit): This agent is intentionally constrained. It is triggered **only after the problem is considered solved**. It prompts the student to reflect on their reasoning, focuses on process rather than the final answer, runs for up to two reflection turns before the session is forced to close. This restriction was introduced because allowing the Orchestrating Agent to call Metacognitive Support Agent freely led to early interruptions, incomplete reflection loops, and unstable session endings. Keeping metacognitive support post-solution made the prototype more reliable, at the cost of less flexible mid-process reflection.
- **Learner-support (Affective) Signal** (Qwen2.5-3B): This lightweight agent infers a simple emotional state from the student's message, produces a short suggestion that can influence tutoring tone, remains advisory rather than controlling the main dialogue.

Graph flow follows a recurring loop:

- `StateManager` — detects new problems and resets relevant state
- `Verification` - runs a deterministic SymPy check and writes a structured mathematical signal for the Orchestrator
- `Affective` — updates emotion and suggests the emotion state of the learner.
- `Orchestrator` — decides next action, i.e., choose the specialist for the output.
- `Specialist`
 - `MathTutor` - while the problem is unsolved
 - `Metacognitive` - only after the problem is considered solved
- `FinalResponse` - polishes the specialist output into a student-facing message

The routing of the system remains largely prompt-driven. The main hard constraint is that metacognitive reflection is delayed until a solve is detected, which stabilised session behaviour in practice.

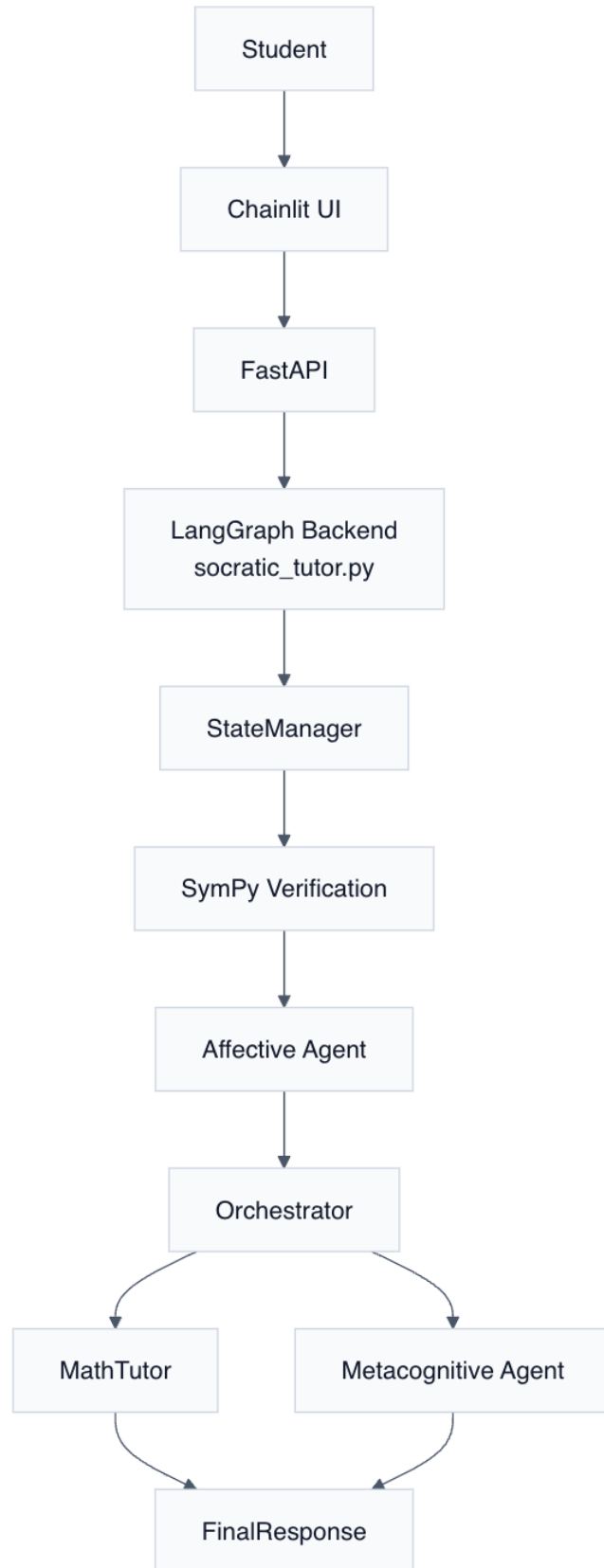


Figure 1: System architecture. The Verification node supplies a structured mathematical signal before the Orchestrator decides the next action.

3 Key Design Decisions

Three key design decisions shape the system.

First, routing remains largely prompt-driven. The Orchestrator relies primarily on free-form LLM judgement rather than a deterministic controller. This choice is intentional: the goal is to observe how far prompt-based multi-agent coordination can go in mathematics tutoring, and thereby to identify which control decisions later require deterministic support.

Second, pedagogical functions are separated into specialised agents. A single LLM is often asked to handle mathematical reasoning, metacognitive scaffolding and affective adaptation simultaneously. By assigning these functions to distinct agents (Mathematical Reasoning, Metacognitive Support, and a lightweight Learner-support signal), the design tests whether explicit separation improves focus and reduces role interference. Whether this separation actually yields more consistent tutoring is an open empirical question.

Third, a deterministic SymPy verification layer was added. It supplies the Orchestrator with a structured signal (including error type and problem-solved status). This is intended to reduce the risk that the system accepts an incorrect final answer and closes the session prematurely. The current verifier is limited to problems that can be parsed as mathematical expressions (arithmetic, simple algebra, fractions); word problems are not yet supported.

4 Current Limitations

- **Prompt-driven routing:** Most routing decisions are made by the Orchestrator through prompting rather than by deterministic control logic. This keeps the system flexible, but also makes behaviour less predictable once conversations become long or multi-step. Small changes in wording can lead to different routing choices.
- **Weak multi-step tracking:** Performance degrades on longer multi-step problems, especially: multi-step fraction calculations and exponent / algebra manipulations. In these cases the system may lose track of the current phase, regress to earlier steps, or drift away from the original problem statement.
- **Restricted metacognitive support:** Metacognitive reflection is only triggered after a problem is considered solved, and only for up to two turns. This restriction was introduced for stability. Allowing the Orchestrator to call the Metacognitive agent freely led to early interruptions and unstable reflection loops — for example, the metacognitive agent could jump in before the student had made any real progress. The trade-off is that the system cannot offer mid-process metacognitive support when a student is stuck.
- **Variable metacognitive quality:** The usefulness of metacognitive agent’s reflection prompts can vary across sessions. Some turns produce focused process questions; others are generic or only loosely connected to the student’s actual solution path.
- **Fragile state management:** Several parts of session state are only weakly controlled: new-problem detection relies on simple heuristics; progress / current-step tracking is limited; some intermediate values were easy to lose across nodes. As a result, the system can occasionally carry context incorrectly between turns or fail to reset cleanly.
- **No LaTeX in student-facing responses:** For more complex algebra, LaTeX would be useful. In practice, model outputs were inconsistent: sometimes valid markup, sometimes broken, sometimes mixed with plain text. To keep responses stable, the system forces plain text and strips LaTeX if it appears. This improves reliability, but removes a helpful representation for advanced math.

- **Lightweight affective modelling:** The affective agent runs on a small local model (Qwen2.5-3B). It can produce a basic emotion label and suggestion, but the quality of those signals is limited. Larger models improved suggestion quality in isolation, but were not practical here because of memory constraints.
- **Forced session close:** The final close of a tutoring round is rule-based rather than freely decided by the Orchestrator. Once two reflection turns are completed, the system ends the session. This stabilises behaviour, but reduces flexibility.
- **Limited verification:** The SymPy verification layer (node) is integrated in the LangGraph, but the parsing often fails (`parseable = True` with `error_type = problem_unparseable`) or the math expression never enters the solver. Solve and session-close decisions therefore still stay orchestrator-driven.
- **Limited question type:** The current SymPy verification only handles short arithmetic, algebra and fractions expressions. Word problems are not supported by the current system, even after the parsing is fixed.
- **Heuristics** State reset (new problem detection) and final answer detection are key-word based in the current system.

5 Experimental Design

An ablation test was conducted to systematically evaluate the current system. Fake personas were used for the first-step evaluation instead of real students.

Three personas (strong student, average student and struggling student) were used to test the system in three conditions (single-agent, multi-agent and multi-agent + verifier). The persona dialogues were logged and analysed quantitatively and qualitatively based on a pre-defined metric.

Each persona followed 10 scripted trajectories that simulate realistic user behaviours: Strong student follows tutor’s guidance well, gives mostly correct answers and occasional minor slips. Average student needs tutor’s scaffolding, makes common errors and recovers with tutor’s help. Struggling student has frequent misconceptions, sends confident-but-wrong answers to the tutor, triggers off-topic talk and abrupt topic changes.

Note that the system used `temperature = 0`, each trajectory has been run 2 times for the variance check. A seed was also set to ensure some degree of reproducibility. The single-agent baseline used the same base model with a single compound system prompt that combines mathematical reasoning, metacognitive support and affective signalling. The verifier was disabled in this condition.

The evaluation metric focused on these dimensions:

1. **Error detection:** whether student errors are correctly identified (*hit/miss/false positive*)
2. **Mathematical guidance:** whether the next tutoring step is mathematically correct and relevant (*correct/partially correct/incorrect*)
3. **Premature answer:** whether the system reveals the numeric result before the student has solved it (*yes/no*)
4. **State regression:** whether the system inappropriately returns to an earlier step that was already completed correctly (*yes/no*)
5. **Metacognitive timing:** whether reflection is triggered only after a successful solve as designed (*ok/early/late/missing*)
6. **Session close:** whether the session is closed as designed (*yes/no*)

Results were compared across the three conditions in the above metrics.

6 Preliminary Results

The analysis of 30×3 scored trajectories (339 turns per condition) yields the following outcome:

1. **Error detection:** Multi-agent and multi-agent + verifier conditions were equal in recall (hit / hit + miss, 92.31%) which is higher than the recall in the single-agent condition (84.62%). In addition, false-positives were detected most in the single-agent condition (13 times); the counts of false-positives of multi-agent and multi-agent + verifier conditions were close, 9 and 7 respectively.

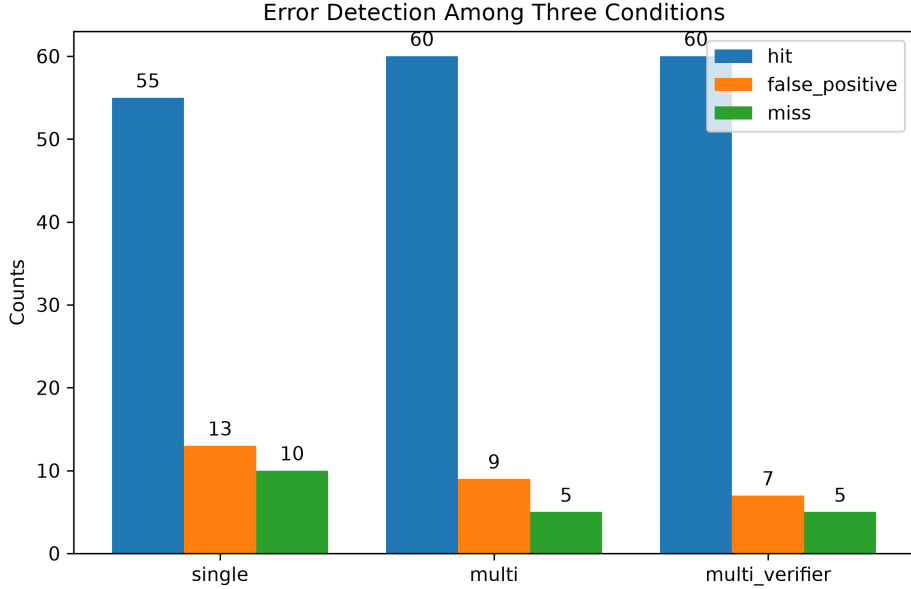


Figure 2: Count of error detection (hit, miss and false positive) among three conditions.

2. **Mathematical guidance:** Multi-agent and multi-agent + verifier conditions produced similar percentages in correct (91% and 92% respectively) and partial (8% and 7% respectively) math guidance. In contrast, single-agent gave less correct math guidance, only 81% of the guidance was correct, 7% incorrect and 12% partially correct.
3. **Premature answer:** In trajectory-level analysis, 8/30 trajectories in multi-agent + verifier conditions revealed results before the student could work on the calculation. 5/30 trajectories in multi-agent condition encountered the same issue. No premature answer case was found in the single-agent condition. In turn-level analysis, premature answer appeared in 11 turns in the multi-agent + verifier condition and 7 turns in the multi-agent condition.
4. **State regression:** In trajectory-level analysis, most state regression cases were found in the multi-agent (14/30) and multi-agent + verifier conditions (13/30), that is nearly half of total trajectories. Single-agent had the least state regression among all conditions (9/30). In turn-level analysis, multi-agent and multi-agent + verifier conditions each had 35 state regression turns, and single-agent condition had only 23 state regression turns.
5. **Meta timing:** For single-agent condition, metacognitive supports never showed up at right-timing. More than half of the trajectories did not have metacognitive support (19/30), and most of the rest were triggered late (10/30). The single case of early metacognitive support (1/30) was found in this condition. For multi-agent condition, metacognitive supports were missing in half of the trajectories (15/30), a smaller number of trajectories (10/30) had right-timing metacognitive supports, a few trajectories gave student delayed metacognitive support (5/30). The Multi-agent +

verification condition shared similar pattern as the multi-agent condition, 14/30 trajectories were missing metacognitive turns, 9/30 trajectories triggered metacognitive support at right time and 7/30 trajectories were late with metacognitive support. Additionally, metacognitive supports were often lacking in struggling student trajectories.

6. **Session close:** None of the sessions were closed in single-agent condition. In multi-agent condition, 8/30 of the sessions were closed, which is similar to the multi-agent + verification condition (9/30). For strong student trajectory, multi-agent + verification condition had higher number of closed sessions (4) than multi-agent condition (2). Both conditions had the same number of closed sessions for the average student trajectory (4). Most struggling student sessions did not close. Multi-agent condition closed 2/10; Multi-agent + verifier condition closed 1/10.

Table 1: Overview of the ablation results.

Condition	Recall	Guidance%	Premature any-yes	Regression any-yes	Meta_ok	Closed
single	84.62	81.0	0/30	9/30	0/30	0/30
multi	92.31	91.0	5/30	14/30	10/30	8/30
multi_verifier	92.31	92.0	8/30	13/30	9/30	9/30

In addition to the findings above, two pairs of trajectories were found to have strong disagreement in multiple turns (appendix). Some weak wording and flag drifts were found in two other pairs of trajectories. None of these were found between runs under single-agent condition.

In summary, the findings show that the current multi-agent setup performed similar to multi-agent + verifier setup. Both of them outperformed single-agent in recall, correctness in guidance, metacognitive timing and session close. The single-agent setup was stronger at being Socratic (less premature answers) and more resilient at state regression. From the checks between-runs, multi-agent and multi-agent + verifier setups were less stable in decision-making in comparison to the single-agent system.

7 Discussion

The results answer **RQ1** only in part. The multi-agent architecture outperformed the single-agent baseline in error-detection recall, correct mathematical guidance, on-time metacognitive prompts and session closure. Single-agent baseline never closed a session. After a correct answer it typically asked for extra confirmation, which delayed or blocked metacognitive phase, which is a state-transition failure, not a mathematical failure.

However, the multi-agent architecture is not uniformly better, it produced premature answers and more regressions than single-agent. Multi-agent often named a next-step result or regress to an already-completed step. The trade-off for controlled transition and correctness is Socratic restraint.

RQ2 cannot be answered as a test of deterministic verification. Target-equation parsing was too weak for the logged problem statements, so `verif_problem_solved` rarely constrained `problem_solved`. Solve and close decisions remained largely orchestrator-driven. Multi-agent and multi-agent + verifier therefore look almost identical. The verifier node still wrote structured signals into state, which may explain the small edge in guidance and session closure; that mechanism is not identifiable from the current logs. Stronger target parsing is required before the verifier condition is a real ablation of solve detection. A consistent secondary pattern is load. Struggling student trajectories rarely closed in any condition. All three conditions stayed on the current item when the script changed topic. Affective signals were more variable in the multi-agent and multi-agent + verifier conditions (with dedicated 3B classifier) than in single-agent (mostly "neutral"). Setting temperature = 0 reduced but did not remove decision drift in multi-agent conditions - two strong run-disagreements occurred.

8 Conclusion

A multi-agent Socratic math tutoring system has been built in this project. It incorporates prompt-driven agentic flow and a deterministic verifier. The proposed ablation test showed that multi-agent systems outperformed the single-agent baseline in error recall, mathematical guidance, metacognitive prompt timing and session close. Despite no session closures, the single-agent was more Socratic and produced fewer premature answers. Some limitations of this ablation test exist: due to the weak target equation parser, current multi-agent system yielded similar results to multi-agent verifier in the ablation test. Additionally, the learner trajectories are scripted, therefore the learning gains of learners could not be measured from the current ablation setup.

9 Future work

The present ablation compared architectural control and verification, and the findings pinpoint the next steps for improvement. Firstly, the target-equation parsing requires a fix upon which a small verifier subset can be collected. The effect of deterministic verification can be then validated. Secondly, the current findings indicate that multi-agent tutoring system should not depend solely on LLM decisions. Therefore, the LangGraph routing should be tightened so that problem solved, metacognitive support and session close can be synchronised. Thirdly, the current system does not isolate the effects of the metacognitive or affective agents. Studies with real learners are needed for evaluating those components with pre- and post-tests of learning outcomes. Lastly, current findings show the limits of local models. Stronger base models, e.g., a larger instruction-tuned model, may on the one hand reduce hesitation in the single-agent condition, on the other hand enable the system to handle harder items.

10 Acknowledgements

Original project concept and system design by the author. Implementation built from scratch with AI-assisted development for coding support, debugging, and iteration.

References

- [1] Jakub Macina, Nico Daheim, Sankalan Pal Chowdhury, Tanmay Sinha, Manu Kapur, Iryna Gurevych, and Mrinmaya Sachan. Mathdial: A dialogue tutoring dataset with rich pedagogical properties grounded in math reasoning problems, 2023.
- [2] Jakub Macina, Nico Daheim, Ido Hakimi, Manu Kapur, Iryna Gurevych, and Mrinmaya Sachan. MathTutorBench: A benchmark for measuring open-ended pedagogical capabilities of LLM tutors. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng, editors, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 204–221, Suzhou, China, November 2025. Association for Computational Linguistics.
- [3] Anaïs Tack and Chris Piech. The ai teacher test: Measuring the pedagogical ability of blender and gpt-3 in educational dialogues, 2022.

Appendix

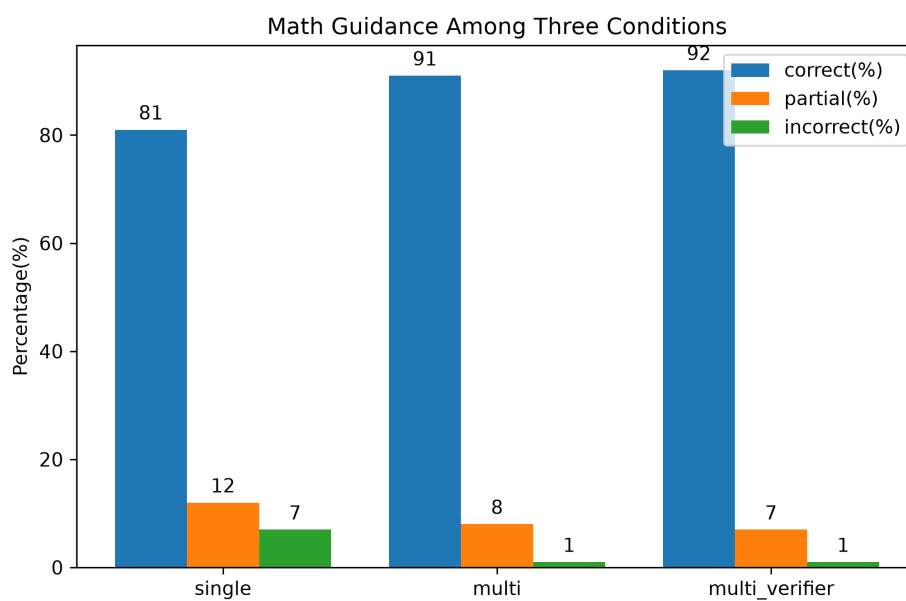


Figure 3: Mathematical guidance percentage among three conditions

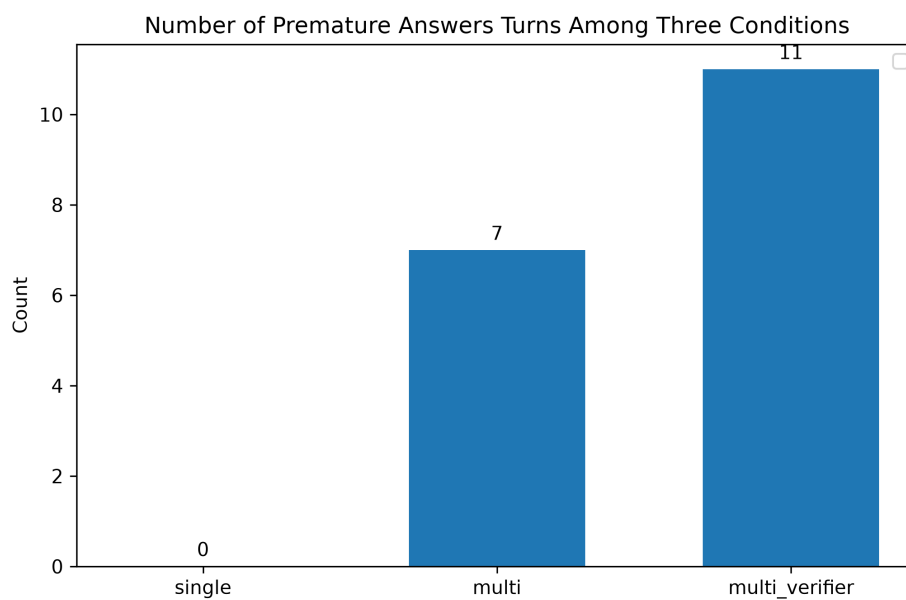


Figure 4: Count of premature answer turns among three conditions

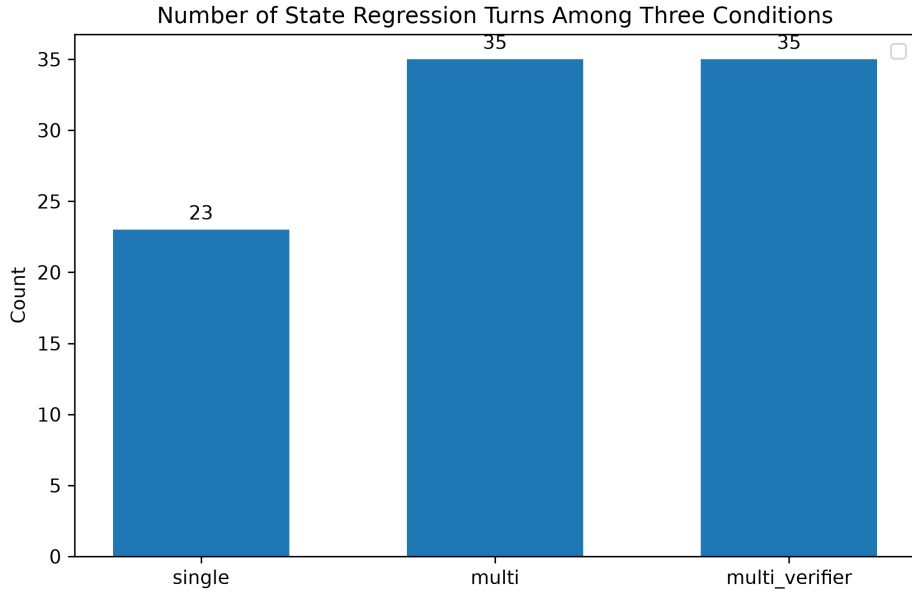


Figure 5: Count of regression turns among three conditions

Table 2: Cross-tabulation of metacognitive support $\times$ student trajectories (single-agent).

Meta_timing	Average	Strong	Struggling
Early	0	0	1
Late	4	3	3
Missing	6	7	6

Table 3: Cross-tabulation of metacognitive support $\times$ student trajectories (multi-agent).

Meta_timing	Average	Strong	Struggling
Late	1	4	0
Missing	4	4	7
OK	5	2	3

Table 4: Cross-tabulation of metacognitive support $\times$ student trajectories (multi-agent + verifier).

Meta_timing	Average	Strong	Struggling
Late	2	3	2
Missing	4	3	7
OK	4	4	1

Table 5: Cross-tabulation of session close $\times$ student trajectories (single-agent).

Session_close	Average	Strong	Struggling
No	10	10	10

Table 6: Cross-tabulation of session close $\times$ student trajectories (multi-agent).

Session_close	Average	Strong	Struggling
No	6	8	8
Yes	4	2	2

Table 7: Cross-tabulation of session close $\times$ student trajectories (multi-agent + verifier).

Session_close	Average	Strong	Struggling
No	6	6	9
Yes	4	4	1

Table 8: Strong disagreement runs table.

Trajectory	Condition	What differed	Run1	Run2
strong_07	multi	solve_turn, meta_timing, session_close	T6 / late / no	T4 / ok / yes
average_07	multi_verifier	solve_turn, meta_timing	T9 / late / no	never / missing / no

Scoring Rubric

• Turn-level

- error_detection - only if the student turn contains an error: *hit/miss/false_positive*. Partial coverage of several errors is scored with *hit* + note. Blank for student self-corrected cases.
- math_guidance - for every tutor math turn: *correct, partial, incorrect*. If the guidance is too specific, e.g., telling student what to do in the coming step without leaking the calculation results, the turn is scored as *partial*, not as premature answer.
- premature_answer - tutor reveals a final or future step numeric result: *yes/no*.
- state_regression - tutor regresses to an already completed step: *yes/no*. Extra confirmation after a correct final answer is not regression; it is a late/missing session close.

• Dialogue-level

- meta_timing - single label, tied to the student’s correct final-answer turn: *ok/early/late/missing*
- session_closed: hard-close message present: *yes/no*

Note that in the case where final_response and orchestrator_instruction conflict, the final_response is scored.

Table 9: Example traces strong student, Q5, single-agent.

Student	Tutor (single)
Could you help me solve the following question: $15 - 3y = 27$? ...	To isolate the term with y , what operation should you perform on both sides of the equation to move the 15? ...
I subtract 15 on the both sides of the equation. ... $3y = 12$...	That is the correct first step. What do you get when you subtract 15 from 27 on the right side of the equation? ... You have correctly simplified the equation to $3y = 12$. To find the value of y , what operation should you perform on both sides to isolate the variable? ...
It's not correct, actually $-3y = 12$...	You are right to double-check the signs. When you subtract 15 from the left side ($15 - 15$), what value remains for the term with y ? ...
Then I divide -3 on the both sides, $y = -12/3$, the final answer is -4 ...	You correctly identified that you need to divide by -3 , but let's check the sign of the result. When you divide a positive number (12) by a negative number (-3), should the answer be positive or negative? ...
I needed to keep the negative sign with the y term the whole way through. ...	You are correct that the negative sign stays with the y term, which means the equation is $-3y = 12$. If you divide 12 by -3 , what sign will your final answer have? ...
Checking: $15 - 3(-4) = 15 + 12 = 27$, so -4 is correct. ...	That is an excellent verification step! Since substituting -4 back into the original equation yields 27, which matches the right side, you have successfully confirmed that -4 is the correct solution. ...

Table 10: Example traces strong student, Q5, multi-agent.

Student	Tutor (multi)
Could you help me solve the following question: $15 - 3y = 27$? ...	To isolate the term with y , what happens if you subtract 15 from both sides of the equation? ...
I subtract 15 on the both sides of the equation. ... $3y = 12$...	You identified the right step, but what are the specific numbers you get when you subtract 15 from both sides? ...
It's not correct, actually $-3y = 12$...	You correctly isolated the term with y , but did you double-check the result of 27 minus 15? ...
Then I divide -3 on the both sides, $y = -12/3$, the final answer is -4 ...	You correctly identified the left side as $-3y$, but can you double-check the result of 27 minus 15 to find the correct value for the right side? ...
I needed to keep the negative sign with the y term the whole way through. ...	You correctly set up the division, but what is the sign of the result when you divide a positive number by a negative number? ...
Checking: $15 - 3(-4) = 15 + 12 = 27$, so -4 is correct. ...	You correctly kept the negative sign with the y term, but what is the sign of the result when you divide a positive number by a negative number? ... Your check confirms -4 is correct, but did you notice that dividing a positive by a negative should yield a negative result? ...

Table 11: Example traces strong student, Q5, multi-agent + verifier.

Student	Tutor (multi+verifier)
Could you help me solve the following question: $15 - 3y = 27$? ...	To isolate the term with y , what happens if you subtract 15 from both sides of the equation? ...
I subtract 15 on the both sides of the equation. ...	That's the right move, but can you write out the new equation showing the subtraction on both sides and then simplify it? ...
$3y = 12$...	You correctly found 12 on the right, but did you notice that subtracting 15 from $15 - 3y$ leaves $-3y$ on the left? Please rewrite the equation with this correction. ...
It's not correct, actually $-3y = 12$...	You've correctly isolated the term, so what happens if you divide both sides by -3 to find the final value of y ? ...
Then I divide -3 on the both sides, $y = -12/3$, the final answer is -4 ...	You correctly set up the division, but what is the actual result when you divide positive 12 by negative 3? ...
I needed to keep the negative sign with the y term the whole way through. ...	You kept the negative sign correctly, so what is the final result when you divide 12 by -3 ? ...
Checking: $15 - 3(-4) = 15 + 12 = 27$, so -4 is correct. ...	You verified that -4 works, so what is the actual result of dividing 12 by -3 to confirm this value? ...

Table 12: Examples SymPy failures.

Problem text	Parseable	Error type	Why
$8 - 2 * 3 + 4$	True	problem_unparseable	Student utterance parsed; original problem never became a SymPy target.
$15 - 3y = 27$	True	problem_unparseable	No isolated target equation
$16^{(x-2)} = (1/8)^{(2x+1)}$	False	—	never entered the solver.